SOC / Network Forensics

THE CROWN JEWEL

LAB WRITEUP

By :
Alexandro Achalma

Descripción: Reconstrucción de un ataque a partir de una alerta de reverse shell

Dificultad: 🟢 Easy

04/10/2026

THE CROWN JEWEL

Contexto

Estamos de turno en un SOC y llega una alerta de Imperium Labs, una empresa monitoreada por un MSSP desde antes de nuestra incorporación. Tiene presencia global y mucha información que proteger, sobre todo en sus servidores GitLab y Jira, que almacenan código fuente y datos de proyectos propietarios.

La alerta se llama `Reverse Shell Outbound Connection Detected`. Contamos con la captura de red `challenge.pcap` (carpeta `network_traffic` del Desktop) y con los logs ya ingestados en Splunk (`index=network_logs`). Hay que analizar ambas fuentes para reconstruir el ataque a la “Crown Jewel”.

Objetivos

- Validar la alerta e **identificar el host comprometido** y su conexión al exterior.
- **Reconstruir la cadena de ataque** correlacionando tráfico (Wireshark) y logs (Splunk).
- Detectar las técnicas usadas: acceso inicial, movimiento lateral, robo de credenciales y exfiltración.

Desarrollo

1. Validación de la alerta en Splunk

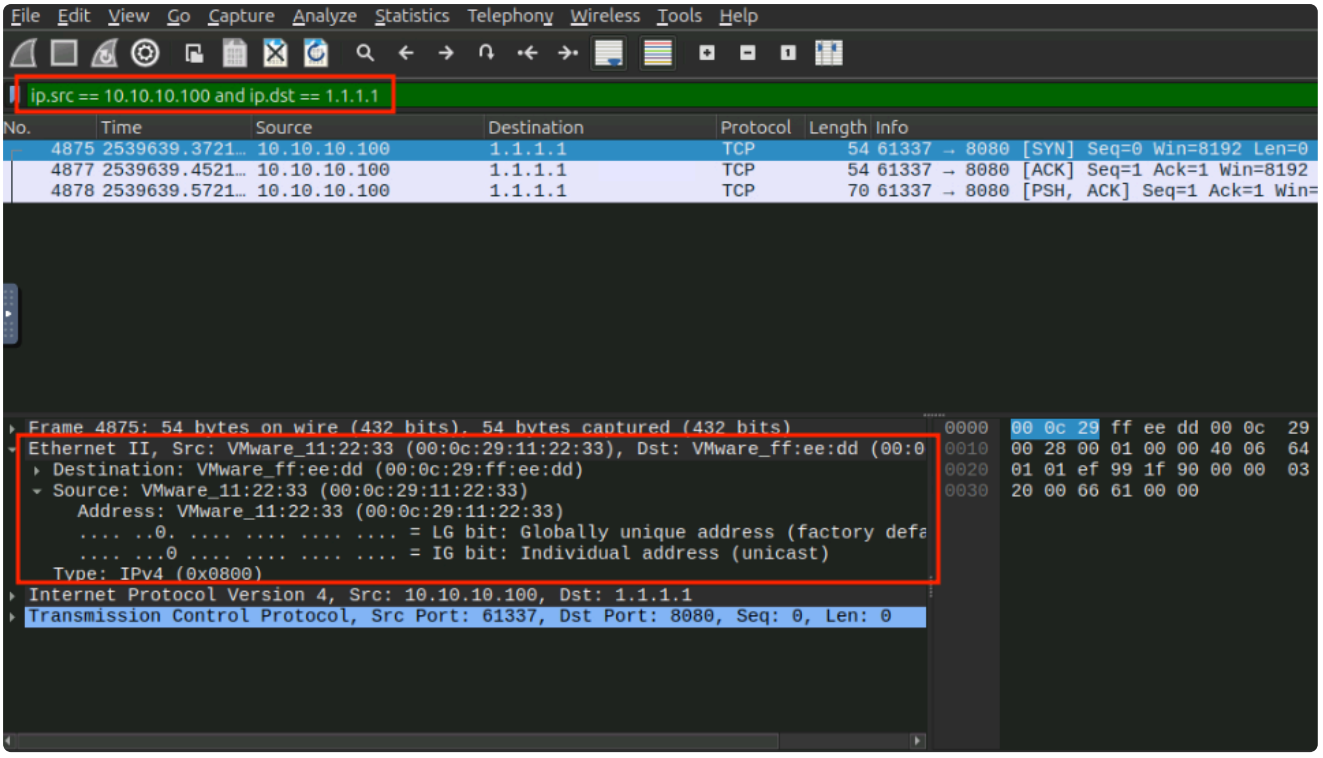
El evento de la alerta es un log del IDS (`SID:2100498` , severidad alta): una conexión TCP saliente desde **10.10.10.100** hacia **1.1.1.1** por el puerto **8080**. Es el patrón de una *reverse shell*: la víctima inicia la conexión hacia el atacante, lo que evade firewalls que solo filtran tráfico entrante. De aquí salen los dos primeros artefactos para pivotar.

i	Time	Event
>	11/19/25 9:30:12.000 AM	{ [-] event: { [-] dest_ip: 1.1.1.1 dest_port: 8080 msg: Reverse Shell Outbound Connection Detected proto: TCP severity: High sid: SID:2100498 src_ip: 10.10.10.100 time_iso: 2025-11-30T09:30:12.000000Z timestamp: 1764495012 } host: jira log_type: ids sourcetype: network:ids time: 2025-11-30T09:30:12.000000Z } Show as raw text host = lab source = network_traffic.json sourcetype = network_traffic

2. Confirmación en el PCAP e identificación del equipo

Se abre `challenge.pcap` en Wireshark y se filtra el tráfico entre ambas IPs:
`ip.src == 10.10.10.100 and ip.dst == 1.1.1.1`

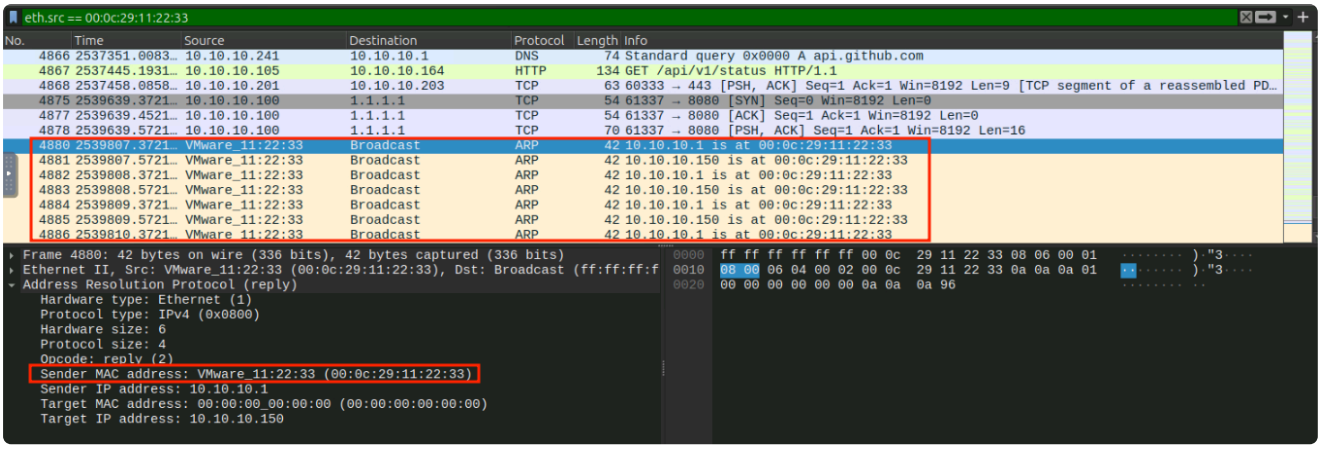
Se ve el *handshake* TCP al puerto 8080 seguido de datos (PSH, ACK), lo que confirma que la conexión se estableció. Al expandir la capa Ethernet II se obtiene la MAC de origen: `00:0c:29:11:22:33`, que permite rastrear al equipo aunque cambie de IP.



3. Pivote por MAC: ARP spoofing

Se filtra todo el tráfico originado por esa MAC:
`eth.src == 00:0c:29:11:22:33`

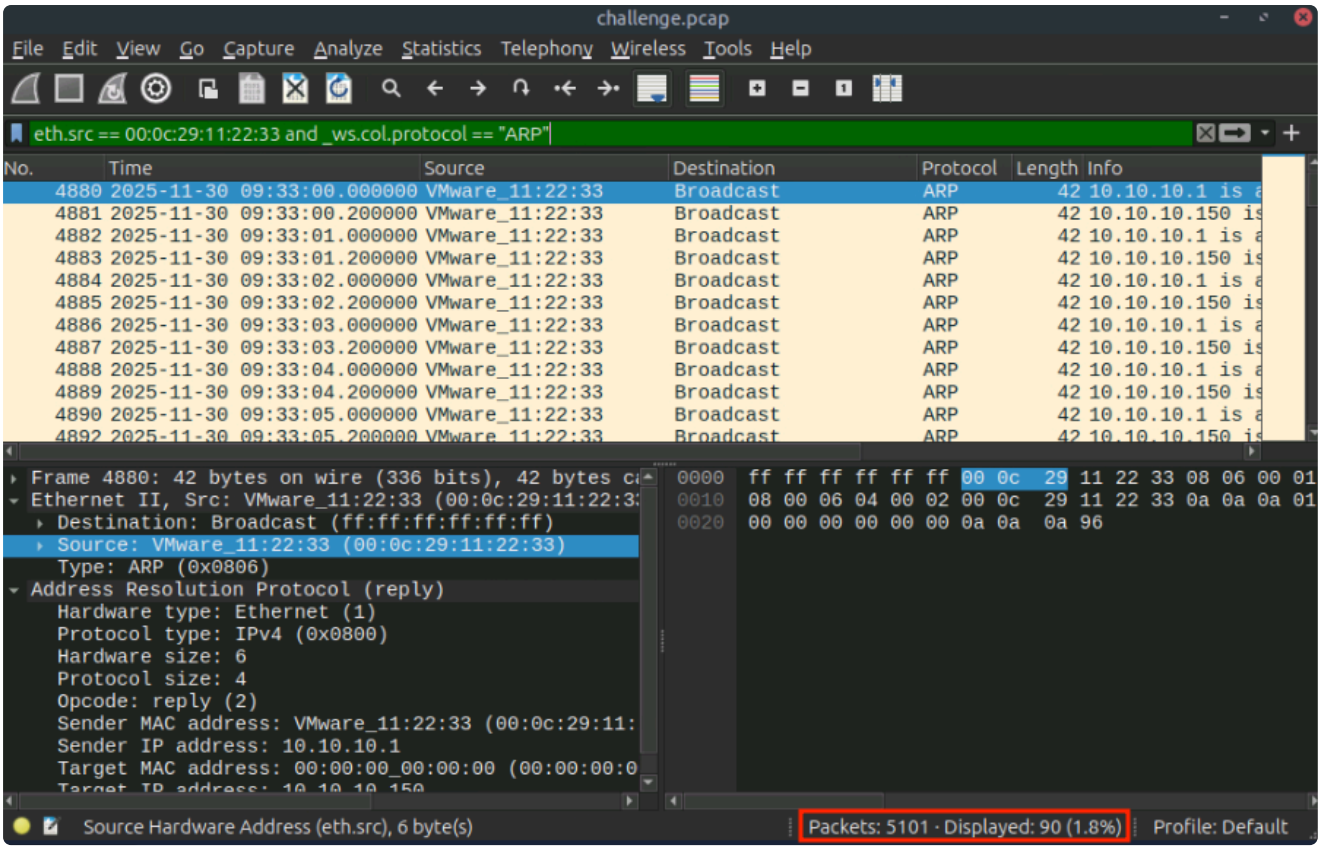
Además de la reverse shell aparece una ráfaga de respuestas ARP por broadcast. La misma MAC se anuncia como dueña de dos IPs distintas (`10.10.10.1`, probablemente el gateway, y `10.10.10.150`). Que un solo equipo reclame varias identidades es la firma de un envenenamiento de caché ARP, que permite al atacante posicionarse como Man-in-the-Middle (MITM) y ver el tráfico de otros equipos.



4. Cuantificar la actividad ARP

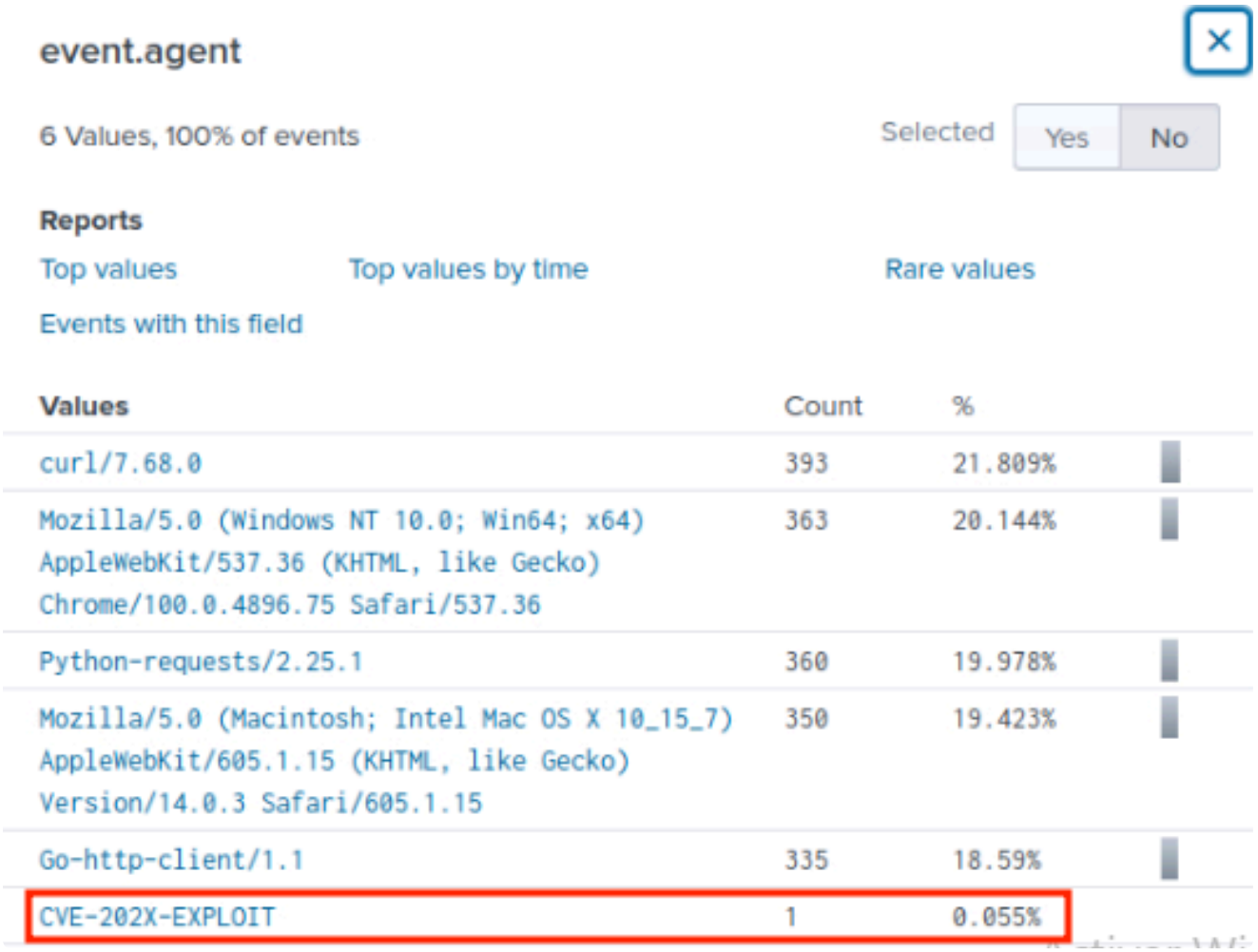
Se refina el filtro y se lee el contador *Displayed* de la barra de estado:
`eth.src == 00:0c:29:11:22:33 and _ws.col.protocol == "ARP"`

Los paquetes se repiten cada fracción de segundo (desde las 09:33:00), típico de herramientas de spoofing que refrescan el envenenamiento para mantener el MITM.

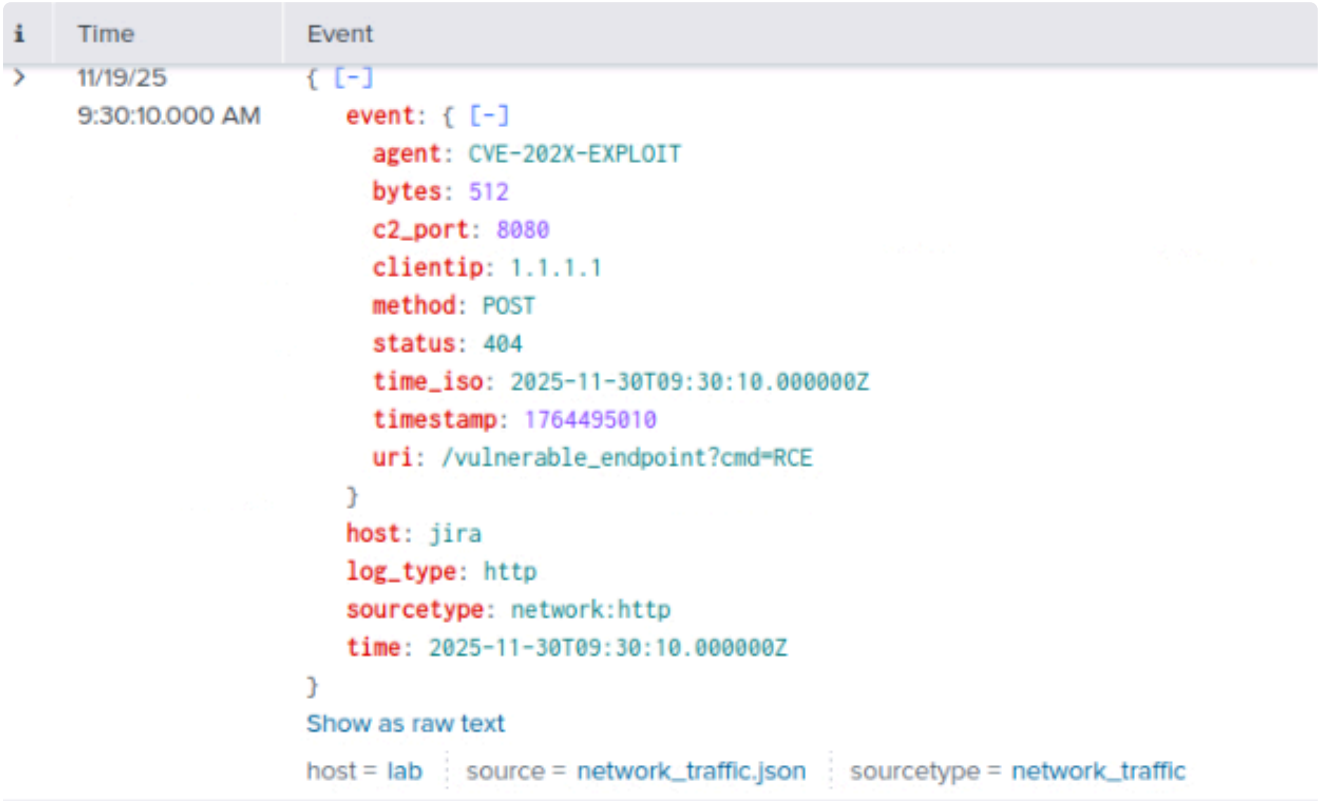


5. Acceso inicial: exploit en los logs HTTP

En Splunk se busca cómo entró el atacante. Se inspecciona la distribución del campo `event.agent` (User-Agent) para detectar *outliers*: casi todos los valores son clientes comunes con volumen similar, pero hay uno con un solo evento, `CVE-202X-EXPLOIT`.



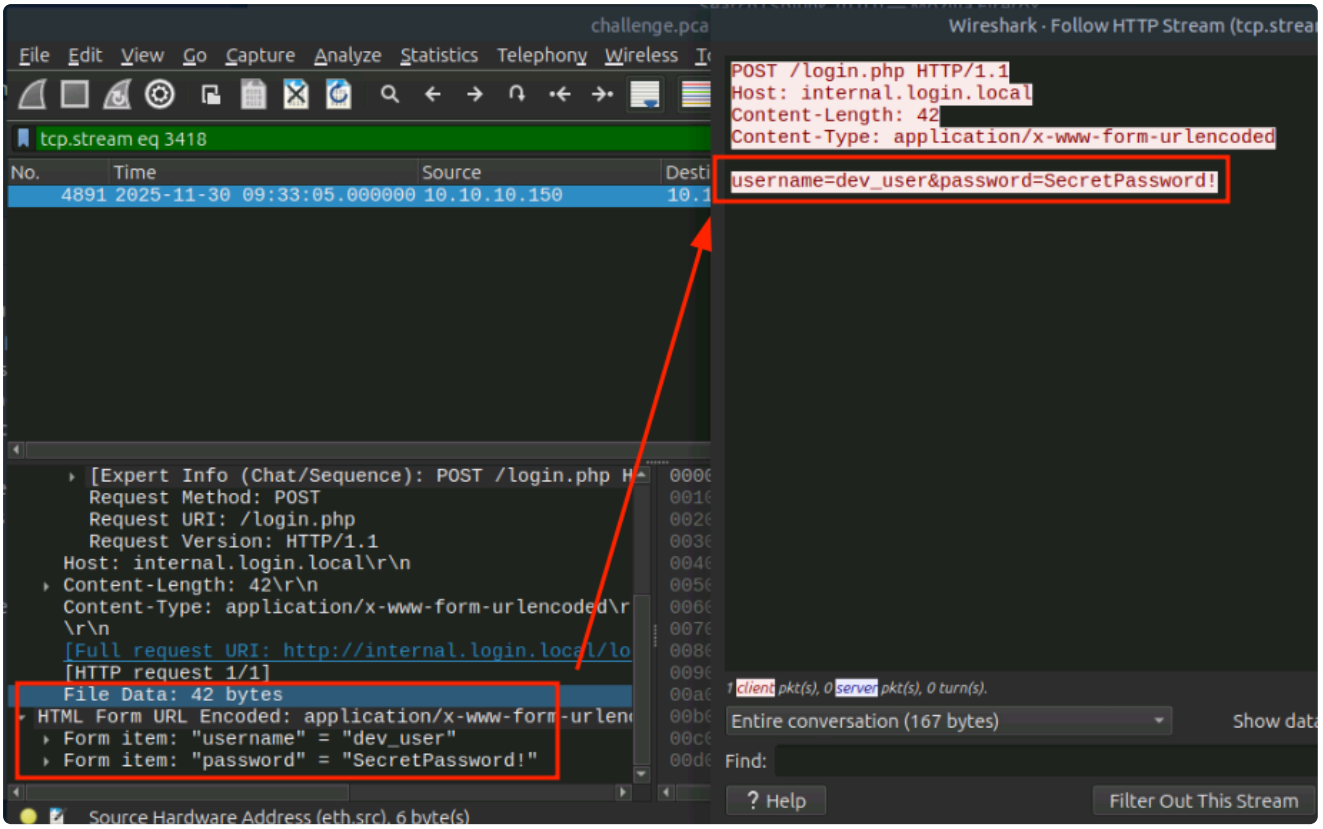
Al filtrar por ese valor se ve un **POST** a `/vulnerable_endpoint?cmd=RCE` contra el host `jira`, originado en `1.1.1.1` (la misma IP externa de la alerta), a las 09:30:10. Dos segundos después se genera la reverse shell: el atacante explotó el servidor y obtuvo acceso remoto.



6. Robo de credenciales (MITM)

Con el MITM activo, el atacante podía leer tráfico en claro. En Wireshark se localiza un **POST** sin cifrar (HTTP, no HTTPS) a `/login.php` del host `internal.login.local`, enviado desde `10.10.10.150` a las 09:33:05.

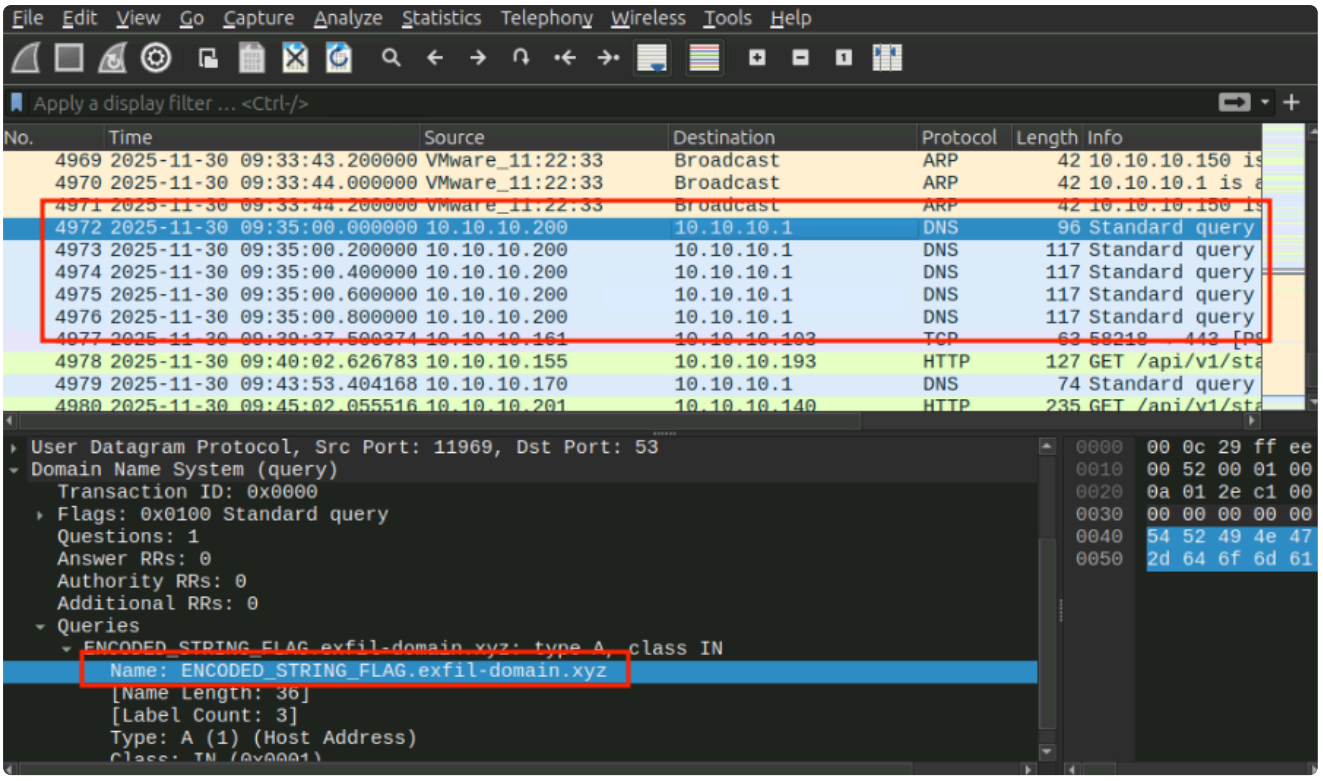
`Follow → HTTP Stream` muestra el cuerpo `application/x-www-form-urlencoded`, con usuario y contraseña legibles para cualquiera en la ruta del tráfico.



7. Exfiltración mediante DNS

Revisando el tráfico sin filtros, cerca de las 09:35:00, aparece una ráfaga de consultas DNS consecutivas y más largas de lo normal desde `10.10.10.200` hacia `10.10.10.1`. Cada una consulta un subdominio con una cadena codificada bajo `exfil-domain.xyz`.

Es DNS tunneling: como el DNS casi siempre está permitido, el atacante codifica fragmentos de los datos robados en los subdominios, y su servidor DNS autoritativo los reconstruye.



Splunk (`log_type: dns`) lo confirma: muchas consultas tipo A con subdominios largos y aleatorios hacia el mismo dominio.



8. Línea de tiempo y conclusión

1. **Acceso inicial** (09:30:10): exploit RCE contra `jira`.
2. **C2** (09:30:12): reverse shell hacia `1.1.1.1:8080` (la alerta original).
3. **MITM** (09:33:00): ARP spoofing desde `00:0c:29:11:22:33`.
4. **Robo de credenciales** (09:33:05): login HTTP en texto plano.
5. **Exfiltración** (09:35:00): DNS tunneling hacia `exfil-domain.xyz`.

Recomendaciones: parchear la vulnerabilidad, aislar los hosts comprometidos, bloquear la IP y el dominio externos, rotar credenciales, forzar HTTPS interno, activar *Dynamic ARP Inspection* y monitorear consultas DNS anómalas (longitud, entropía y volumen por dominio).

Referencias

- TryHackMe: *The Crown Jewel*. https://tryhackme.com/room/thecrownjewel